

TALLINNA TEHNIKAÜLIKOOL
Infotehnoloogia teaduskond

Igor Ustritski 232908IADB

**Eesti jalgpalliklubide haldamine veebipõhise
rakenduse abil**

Bakalaureusetöö

Juhendaja: Dmitri Mironov

Tallinn 2026

Autorideklaratsioon

Kinnitan, et olen koostanud antud lõputöö iseseisvalt ning seda ei ole kellegi teise poolt varem kaitsmisele esitatud. Kõik töö koostamisel kasutatud teiste autorite tööd, olulised seisukohad, kirjandusallikatest ja mujalt pärinevad andmed on töös viidatud.

Autor: Igor Ustritski

Annotatsioon

Käesolev bakalaureusetöö keskendub Eesti jalgpalliklubide haldusprotsesside digitaliseerimisele veebipõhise rakenduse prototüübi abil. Töös analüüsitakse klubide praeguseid töökorralduse kitsaskohti, võrreldakse olemasolevaid lahendusi ning kavandatakse süsteem, mis koondab liikmete halduse, treeningajakavade koordineerimise ja klubisisese teavitamise ühtsesse keskkonda. Prototüübi arhitektuur põhineb ees- ja tagarakenduse eraldamisel ning rollipõhisel ligipääsukontrollil. Tehniline teostus kasutab Java Spring Boot tagarakendust, React eesrakendust ning PostgreSQL andmebaasi.

Abstract

Football Club Management Using a Web-Based Application

This bachelor's thesis addresses the digitalization of administrative processes in Estonian football clubs through a web-based application prototype. The study analyses current organizational challenges, compares existing solutions, and designs a system that consolidates membership management, training schedule coordination, and internal communication into a single environment. The prototype architecture is based on a separated front-end and back-end approach with role-based access control, implemented using Java Spring Boot, React, and PostgreSQL.

Lühendite ja mõistete sõnastik

API – Application Programming Interface, rakendusliides

RBAC – Role-Based Access Control, rollipõhine ligipääsukontroll

UI – User Interface, kasutajaliides

REST – Representational State Transfer, veebiteenuste arhitektuurstiil

XSS – Cross-Site Scripting, veebiliidese turvarisk

HA – High Availability, kõrge kättesaadavus

Sisukord

Autorideklaratsioon	2
Annotatsioon.....	3
Abstract.....	4
Lühendite ja mõistete sõnastik	5
Sisukord.....	6
Jooniste loetelu	8
1 Sissejuhatus	9
2 Ülesandepüstitus.....	10
2.1 Probleemi kirjeldus.....	10
2.2 Töö eesmärk ja uurimisülesanne	11
2.3 Uurimis- ja arendusmeetod.....	12
3 Teoreetiline taust ja lähtekohad	13
3.1 Spordiklubide haldus ja tööprotsessid	13
3.2 Infosüsteemide roll haldusprotsesside toetamisel.....	13
3.3 Rollipõhine ligipääs infosüsteemides	14
4 Olemasolevate lahenduste analüüs ja süsteemi nõuded	15
4.1 Analüüsi meetodika ja valikukriteeriumid	15
4.2 Sportlyzer	16
4.3 ProSoccerData	17
4.4 TeamSnap	18
4.5 Analüüsi kokkuvõte ja järeldused.....	19
4.6 Süsteemi funktsionaalsed nõuded.....	20
4.7 Süsteemi mittefunktsionaalsed nõuded	22
5 Süsteemi analüüs ja arhitektuurilised valikud	25
5.1 Süsteemi arhitektuuri ülevaade.....	25
5.1.1 Süsteemi põhivood	25
5.2 Eesarenduse ja tagarakenduse roll.....	27
5.3 Platvormi valik	27
5.3.1 Veebirakenduse sobivus prototüübiks	28
5.3.2 Hübriid mobiilirakenduse piirangud.....	28
5.3.3 Valiku tulemus.....	28
5.4 Tagarakenduse tehnoloogia valik	Error! Bookmark not defined.
5.4.1 Spring Boot (Java).....	Error! Bookmark not defined.

5.4.2 ASP.NET Core (C#).....	Error! Bookmark not defined.
5.4.3 Express.js (Node.js).....	Error! Bookmark not defined.
5.4.4 Tagarakenduse valiku kokkuvõte	Error! Bookmark not defined.
5.5 Eesrakenduse tehnoloogia valik	Error! Bookmark not defined.
5.5.1 React	Error! Bookmark not defined.
5.5.2 Angular	Error! Bookmark not defined.
5.5.3 Vue.js	Error! Bookmark not defined.
5.5.4 Eesrakenduse valiku kokkuvõte	Error! Bookmark not defined.
5.6 Andmebaas ja konteineripõhine keskkond	Error! Bookmark not defined.
5.6.1 PostgreSQL.....	Error! Bookmark not defined.
5.6.2 Docker	Error! Bookmark not defined.
5.7 Tehnoloogiate ja arhitektuuri kokkuvõte.....	Error! Bookmark not defined.
6 Arendustöö	38
6.1 Arendusprotsessi korraldus.....	38
6.2 Andmebaasi mudel	38
6.3 Tagarakenduse realiseerimine	38
6.4 Eesrakenduse realiseerimine.....	39
6.5 Konteineripõhine keskkond.....	39
7 Valminud rakendus ja hindamine	40
7.1 Prototüübi ülevaade	40
7.2 Funktsionaalse sobivuse hindamine	40
7.3 Piirangud ja edasiarendusvõimalused.....	40
Kokkuvõte	42
Kasutatud kirjandus	43

Jooniste loetelu

[TODO] Lisa jooniste ja tabelite loetelu pärast praktilise osa valmimist.

1 Sissejuhatus

Jalgpall on Eestis üks enim harrastatavaid ja jälgitavamaid spordialasid, mille toimimine sõltub suurel määral klubide sisemisest korraldusest ning tõhusast infovahetusest.

Digitaalse ühiskonna areng on suurendanud vajadust tõhusa ja selge infovahetuse järele erinevate organisatsioonide ning nende osapoolte vahel. Spordivaldkonnas, kus tegevused hõlmavad suurt hulka osalejaid ja sündmusi, on ajakohase ja usaldusväärse teabe kättesaadavus oluline nii organisatsioonide toimimise kui ka osapoolte kaasatuse seisukohast.

Eesti jalgpalliklubide igapäevane tegevus hõlmab treeningute ja võistluste korraldamist, võistkondade ja mängijate haldamist ning pidevat suhtlust treenerite, sportlaste ja lapsevanemate vahel. Praktikas kasutatakse selleks sageli mitmeid erinevaid töövahendeid ja suhtluskanaleid, mille paralleelne kasutamine muudab infovahetuse ebaühtlaseks ning raskendab tervikliku ülevaate säilitamist. Selline ebaühtlasus suurendab administratiivset koormust ning võib põhjustada olukodi, kus oluline teave ei jõua osapoolteni õigeaegselt või üheselt mõistetaval kujul.

Käesolev bakalaureusetöö eesmärgiks on uurida, kuidas oleks võimalik neid takistusi ületada veebipõhise rakenduse prototüübi abil, mis koondaks klubi kolm keskset haldusprotsessi, liikmete halduse, treeningajakavade koordineerimise ja klubisisese teavitamise, ühtsesse keskkonda. Töö fookuses on lahenduse kavandamine ja prototüübi realiseerimine, mille põhjal saab hinnata sellise süsteemi praktilist sobivust Eesti jalgpalliklubide igapäevaseks kasutuseks.

2 Ülesandepüstitus

Käesolevas peatükis määratletakse töö keskne probleem, seatakse eesmärk ning kirjeldatakse metoodilist lähenemist.

2.1 Probleemi kirjeldus

[TODO]. Võrrelda välismaa klubidega. Nt eesti suurem jalgpalli kool 1000 last, kõige keskmine hollandi klubi 3000 last ja tippklubides 10000 last. Leida reaalsed faktid ja näidata tulevates peatükkides et väga keerukat veebilehte ei ole vaja

Eesti Spordiregistri andmetel on riigis registreeritud üle 250 jalgpalliklubi, kus osaleb ligikaudu 25 000 harrastajat, mis näitab spordiala laialdast levikut ja ühiskondlikku olulisust [1]. Iga klubi igapäevatöö koosneb mitmest omavahel seotud protsessist: võistkondade moodustamine, treeningajakavade koostamine, väljakute jaotamine ning pidev informatsiooni vahetus treenerite, mängijate ja lapsevanemate vahel. Nende protsesside koordineerimise keerukus kasvab võrdeliselt klubi suuruse ja võistkondade arvuga.

Eesti jalgpalliklubid on oma suuruselt väga erinevad. Käesolevas töös mõistetakse väikese klubina organisatsiooni, kus harrastajate arv jääb alla 100 ning tegutsetakse ühe kuni kahe võistkonnaga. Keskmise suurusega klubiks loetakse organisatsiooni, kus harrastajate arv jääb vahemikku 100 kuni 500 ning tegevus hõlmab mitut vanusegruppi ja võistkonda. Eesti Spordiregistri andmetel on selliseid klubisid Eesti Jalgpalli Liidu 109 liikmesklubi seas valdav enamus [25]. Näiteks on JK Tallinna Kalevi harrastajate arv 24 ja MTÜ Virumaa Jalgpalli harrastajate arv 150, samas kui JK Tabasalu ja JK Paide Linnameeskonna harrastajate arv ulatub ligi 470ni [25]. Suuremad ja professionaalsemad klubid, kes osalevad Premium liigas, kasutavad üldjuhul spetsialiseeritud haldusplatvorme, mille juurutamine ja ülalpidamine eeldab nii rahalist ressursi kui ka eraldi personali [9], [10]. Väiksemad ja keskmise suurusega klubid, eelkõige Esiliiga ja maakondlike liigade tasandil, tegutsevad oluliselt piiratumate vahenditega ning vajavad lihtsat ja taskukohast lahendust, mis kataks igapäevased põhiprotsessid ilma liigse keerukuseta.

Praeguses olukorras puudub enamikul neist klubidest ühtne digitaalne töökeskkond. Treeningute ajakavasid edastatakse e-kirjade, sotsiaalmeediarühmade või kiirsõnumirakenduste kaudu, väljakute saadavust hallatakse sageli tabelarvutusfailides ning liikmete andmed on jaotunud mitme allika vahel. Selline korraldus tähendab, et treenerid peavad tegema palju käsitsi koordineerimistööd, mis võtab aega sportliku tegevuse arvelt.

Eriti teravalt ilmneb probleem noortejaoskondade puhul. Seal osalevad kommunikatsioonis lisaks treeneritele ja noormängijatele ka lapsevanemad, kelle käest tuleb saada kinnitusi osalemise, transpordikorralduse ja varustuse kohta. Mitmekanalilisus suurendab tõenäosust, et sõnumid lähevad kaduma või jõuavad kohale viivitusega.

Seega seisneb uuritav probleem jalgpalliklubide haldus- ja kommunikatsiooniprotsesside ebäühtluses ning puudulikus süsteemses toes, mis võimaldaks siduda liikmete halduse, ajakavade planeerimise ja ressursikasutuse üheks tervikuks.

2.2 Töö eesmärk ja uurimisülesanne

Käesoleva bakalaureusetöö eesmärk on kavandada ja realiseerida veebipõhise rakenduse prototüüp, mis toetab Eesti jalgpalliklubide põhilisi haldusprotsesse ning koondab need ühtsesse digitaalsesse keskkonda. Prototüüp on suunatud eelkõige väikeste ja keskmise suurusega klubide vajadustele ning arvestab kohalikku keele- ja töökorralduslikku eripära.

Töö praktilise tulemusena valmib prototüüp, mille abil on võimalik testida järgmisi põhifunktsionaalsusi:

- võistkondade moodustamine, liikmete registreerimine ning rollipõhine haldus;
- treeningväljakute broneeringute ja ajakavade keskne koordineerimine klubitasandil;
- klubisisene teavitamine, sealhulgas treeningutele osalemise kinnitamine ning muudatustest informeerimine.

Töö uurimisülesandeks on hinnata, millisel määral suudab loodud prototüüp koondada Eesti jalgpalliklubide kolm põhilist haldusprotsessi ühte keskkonda ning milliseid piiranguid ilmneb selle lahenduse praktilisel rakendamisel.

2.3 Uurimis- ja arendusmeetod

Töö on oma olemuselt arendus- ja disainipõhine uurimus [2], mille keskseks meetodiks on veebipõhise infosüsteemi prototüübi kavandamine, realiseerimine ja hindamine. Lähenemine on valitud seetõttu, et eesmärk ei piirdu probleemi kirjeldamisega, vaid hõlmab ka lahenduse praktilist realiseerimist ja hindamist.

Teoreetiline raamistik tugineb kirjanduse analüüsile infosüsteemide kavandamise, protsessijuhtimise ja spordihalduse valdkondades. Olemasolevate lahenduste hindamiseks kasutatakse kvalitatiivset võrdlevat analüüsi, mille tulemuste põhjal sõnastatakse süsteemi nõuded.

Prototüübi arendamisel järgitakse iteratiivse arenduse põhimõtteid [3], kus analüüs, kavandamine ja realiseerimine moodustavad omavahel seotud tsüklilise protsessi. Prototüübi sobivust hinnatakse funktsionaalse analüüsi kaudu: kas realiseeritud lahendus toetab püstitatud põhiprotsesse ning milliseid kitsaskohti praktiline teostus esile toob. Valminud rakenduse hindamiseks kasutatakse kahte meetodit: funktsionaalsete nõuete vastavuse kontrolli ja kasutajakogemuse hindamist küsimustiku abil.

Funktsionaalsete nõuete vastavuse kontroll põhineb peatükis 4.6 määratletud nõuete süstemaatilisel läbivaatamisel. Iga nõude kohta hinnatakse, kas see on prototüübis täielikult täidetud, osaliselt täidetud või täitmata. Vastavuse protsent arvutatakse täidetud nõuete osakaaluna kõigist nõuetest. See meetod võimaldab objektiivselt hinnata, mil määral vastab valminud rakendus algselt seatud eesmärkidele.

Kasutajakogemuse hindamiseks viiakse läbi küsimustikupõhine uuring, milles osalevad potentsiaalsed lõppkasutajad: treenerid ja lapsevanemad. Küsimustik sisaldab väiteid, mida hinnatakse Likerti 5-pallisel skaalal (1 – „Ei nõustu üldse“ kuni 5 – „Nõustun täielikult“). Väited on jagatud kolme kategooriasse: üldine kasutatavus, funktsionaalne sobivus ja visuaalne disain. Lisaks sisaldab küsimustik avatud küsimusi, mis võimaldavad kasutajatel anda vabas vormis tagasisidet. Küsimustiku tulemuste põhjal arvutatakse kategooriate keskmised hinnangud ja tehakse järeldused rakenduse kasutatavuse kohta. Küsimustik on esitatud lisas X.

3 Teoreetiline taust ja lähtekohad

Käesolev peatükk loob teoreetilise aluse, millele tuginevad järgnevad analüüsi- ja kavandamispeatükid. Käsitletakse organisatsioonilisi, infosüsteemseid ning turbealaseid põhimõtteid, mis on otseselt seotud kavandatava lahendusega.

3.1 Spordiklubide haldus ja tööprotsessid

Spordiklubi toimimist saab käsitleda omavahel seotud haldusprotsesside kogumina, mille eesmärk on tagada sportliku tegevuse järjepidevus ning ressursside sihipärane kasutus. Haldustegevus hõlmab liikmete ja võistkondade andmete haldamist, treeningute ja võistluste planeerimist, ressursside koordineerimist ning organisatsioonisisest infovahetust [4]. Iga nimetatud protsessi muutus mõjutab ka teisi: näiteks väljaku saadavuse muutumine tingib treeningkava ümbertegemise, mis omakorda nõuab kõigi asjaosaliste teavitamist.

Protsessipõhise juhtimise teooria rõhutab, et korralduslike tegevuste tulemuslikkus on otseses seoses infoliikumise kiiruse ja täpsusega organisatsiooni sees [5]. Mida rohkem osapooli on kaasatud ja mida ajakriitilisem on edastatav teave, seda suurem on vajadus selgete reeglite ja ühtsete kanalite järele. Ilma süstematiseeritud lähenemiseta tekib olukord, kus teave on organisatsioonis küll olemas, kuid ei jõua õigel ajal õige adressaadini.

Käesoleva töö ja Eesti jalgpalliklubide kontekstis tähendab see, et kavandatav süsteem peab ühendama liikmete, väljakute ja treeningajakavade halduse ühtsesse keskkonda, kus muudatused kajastuvad reaajas kõigile asjaosalistele. Vastasel juhul jääb alles praegune hajutatud korraldus, kus informatsioon liigub paralleelsete kanalite kaudu ning ülevaade puudub.

3.2 Infosüsteemide roll haldusprotsesside toetamisel

Infosüsteemi saab määratleda kui organisatsioonilist vahendit, mis toetab andmete kogumist, töötlemist, säilitamist ja edastamist eesmärgiga muuta tööprotsessid läbipaistvamaks ja paremini juhitavaks [4]. Haldusprotsesside kontekstis aitab infosüsteem kujundada ühtse töökorraldusliku keskkonna, milles protsesside toimimiseks vajalik informatsioon on koondatud ja ajakohane.

Infohalduses eristatakse kahte põhimõttelist mudelit. Tsentraliseeritud mudel koondab andmed ühte kohta ning tagab, et iga muudatus kajastub kõigile kasutajatele üheaegselt [4]. Hajus ehk detsentraliseeritud mudel lähtub sellest, et andmed paiknevad mitmes kohas ning iga kanal võib sisaldada natukene erinevat versiooni samast teabest [5]. Hajusa mudeli puhul on vastuolude risk oluliselt suurem ning ühtsuse saavutamine nõuab täiendavat koordineerimist. Tsentraliseeritud infohaldus toetab seetõttu paremini protsesside läbipaistvust ning vähendab manuaalse koordineerimise vajadust.

Veebipõhised rakendused on levinumaid viise tsentraliseeritud infohalduse elluviimiseks, sest need võimaldavad ligipääsu üle võrgu ilma täiendava tarkvara paigaldamiseta [6]. See omadus on eriti oluline jalgpalliklubide kontekstis, kus kasutajad (treenerid, lapsevanemad, mängijad) peavad saama informatsioonile ligi erinevatest seadmetest ja asukohtadest. Käesoleva töö raames kavandatav süsteem baseerub just veebipõhisel arhitektuuril, et vähendada praegust hajusat infohaldust ning pakkuda ühtset keskkonda kõigile osapooltele.

3.3 Rollipõhine ligipääs infosüsteemides

Rollipõhine ligipääsukontroll (ingl RBAC, role-based access control) on infosüsteemide kavandamise paradigma, kus kasutajate õigused sõltuvad neile omistatud rollidest, mitte individuaalsetest lubadest [7]. RBAC eeliseks on halduse lihtsus: uue kasutaja lisamisel piisab talle rolli määramisest ning kõik selle rolliga seotud õigused rakenduvad automaatselt.

RBAC rakendamisel lähtutakse minimaalsete õiguste printsiibist, mille kohaselt igale rollile antakse üksnes need ligipääsuõigused, mis on vajalikud konkreetsete ülesannete täitmiseks [8]. See vähendab nii turvariske kui ka kasutajaliidese keerukust, sest igale kasutajagrupile kuvatakse ainult talle asjakohane funktsionaalsus.

Kavandatavas süsteemis on rollipõhine lähenemine hädavajalik, kuna süsteemi kasutavad erineva vastutustasemega osapooled. Klubi administraator haldab kogu organisatsiooni struktuuri, sealhulgas võistkondi, väljakuid ja kasutajaid. Treener näeb ja haldab ainult oma võistkondade koosseise, treeninguid ja teavitusi.

4 Olemasolevate lahenduste analüüs ja süsteemi nõuded

Käesolevas peatükis analüüsitakse olemasolevaid jalgpalliklubide haldamiseks mõeldud tarkvaralahendusi ning hinnatakse nende sobivust Eesti jalgpalliklubide praktiliste vajaduste katmiseks. Analüüsi tulemuste põhjal sõnastatakse kavandatava süsteemi funktsionaalsed ja mittefunktsionaalsed nõuded, mis loovad aluse arhitektuurilistele valikutele ja prototüübi realiseerimisele.

4.1 Analüüsi metoodika ja valikukriteeriumid

Analüüsimiseks valiti kolm spordivaldkonnas kasutatavat halduslahendust: Sportlyzer, ProSoccerData ja TeamSnap. Sportlyzer ja ProSoccerData on Eesti jalgpalliklubide seas kasutusel olevad platvormid, mis võimaldavad hinnata nende praktilist sobivust kohalikus kontekstis. TeamSnap valiti võrdlusesse kui rahvusvaheliselt laialt levinud meeskonnahalduse platvorm, et kõrvutada kohalikke lahendusi üldotstarbelise alternatiiviga. Valiku aluseks oli nende funktsionaalsus, mis on suunatud võistkondade ja treeningute haldamisele [9], [10], [11].

Kõiki kolme platvormi testiti praktiliselt, kasutades tegutseva Eesti jalgpalliklubi treenerikontot. Testimise käigus konsulteeriti aktiivse jalgpallitreeneriga, kelle praktilised kogemused aitasid tuvastada platvormi puudusi, mis tekstipõhisest dokumentatsioonist ei ilmne.

Lahendusi hinnati kahel tasandil. Esiteks teostati kvalitatiivne võrdlev analüüs, kus iga platvormi kirjeldati ja hinnati järgmiste kriteeriumide lõikes:

- pakutav funktsionaalsus jalgpalliklubide kolme põhiprotsessi toetamiseks (liikmete haldus, ajakavade koordineerimine, teavitamine);
- sobivus Eesti jalgpalliklubide töökorralduse ja mastaabiga;
- süsteemi keerukus ja kasutuselevõtu eeldatav raskusaste.

Peatükis 4.5 esitatakse koondvõrdlus binaarsete kriteeriumide alusel (Jah/Ei), mis kajastavad, kas lahendus toetab konkreetset funktsionaalsust või mitte. Kriteeriumid tulenevad peatükis 2.1 kirjeldatud probleemist ning Eesti väike- ja keskmise suurusega jalgpalliklubide praktilistest vajadustest.

4.2 Sportlyzer

Sportlyzer on Eestist alguse saanud spordihalduse platvorm, mis on suunatud treeneritele ja klubidele sportlaste arengu jälgimiseks ning treeningprotsesside planeerimiseks. Süsteem pakub vahendeid sportlaste profiilide haldamiseks, treeningute kavandamiseks ning erinevate sportlike näitajate jälgimiseks [9], [12].

Sportlyzeri tugevuseks on ulatuslik funktsionaalsus sportlaste individuaalse arengu jälgimisel ning treeningute struktureerimisel. Platvorm võimaldab treeneritel koostada individuaalseid treeningplaane ja jälgida mängijate füüsilist valmisolekut. Samuti on hästi lahendatud maksete haldamine – treener saab mugavalt jälgida liikmemaksude laekumist ning saata maksmata arvetest meeldetuletusi. Lisaks toetab platvorm osalemiste kinnitamist, mis lihtsustab treeningutele registreerimise jälgimist.

Platvormi praktilisel testimisel ilmneseid siiski mitmed puudused, mis mõjutavad selle kasutatavust Eesti jalgpalliklubide igapäevatöös. Esiteks puudub ajakava loomisel nädalapõhise korduvuse funktsioon – treener ei saa määrata treeningut korduma igal nädalal samal päeval ja kellaajal, vaid peab iga treeningu eraldi käsitsi sisestama. Eesti jalgpalliklubides, kus treeningud toimuvad tavaliselt kindlatel nädalapäevadel kogu hooaja vältel, tähendab see olulist lisahalduskoormust.

Teiseks tuvastas testimine sõnumisaatmise privaatsusprobleemi: treener näeb platvormi sõnumiürikeses kõiki sõnumeid, mis on saadetud teistelt treeneritelt teistele gruppidele. Sellise lahenduse korral ei ole tagatud gruppipõhise infovahetuse konfidentsiaalsus, mis võib olla problemaatiline näiteks personaalsete tagasisidete korral.

[Joonis X: Sportlyzeri sõnumisaatmise vaade, kus treener näeb teiste gruppide sõnumeid — lisa ekraanipilt]

Eesti väiksemate ja keskmise suurusega jalgpalliklubide vaates võib Sportlyzeri piiranguks osutuda selle keerukus ja lai funktsionaalsus, mis ületab sageli väiksemate klubide praktilised vajadused. Kuna tegemist on universaalse spordiplatvormiga, mitte puhtalt jalgpallile suunatud lahendusega, sisaldab süsteem rohkelt funktsionaalsust, mis jalgpalliklubi igapäevatöös kasutust ei leia – näiteks teiste spordialade spetsiifilised mõõdikud ja treeningmeetodid. Süsteemi kasutamine eeldab kasutajate koolitamist ning järjepidevat

andmete sisestamist, mis suurendab halduskoormust. Lisaks ei ole platvorm selgelt suunatud treeningväljakute ja ajakavade lihtsustatud koordineerimisele, mis on paljude Eesti klubide jaoks keskne probleem.

[Igor: otsida konkreetseid näiteid Sportlyzeri ebavajalikust funktsionaalsusest teiste spordialade kontekstis ja lisada siia viide/ekraanipilt].

4.3 ProSoccerData

ProSoccerData on rahvusvaheline jalgpalliklubidele mõeldud haldus- ja analüüsiplatvorm, mis keskendub eelkõige sportlike andmete kogumisele ja analüüsimisele. Süsteem pakub võimalusi mängude, treeningute ja sportlaste statistika haldamiseks ning toetab professionaalset spordianalüüsi [10].

Lahenduse tugevuseks on põhjalik andmepõhine lähenemine, mis võimaldab klubidel jälgida mängijate arengut ja sooritusi detailsemalt. Platvormi analüütilised võimalused – mängijate statistika, soorituse võrdlus ja arengugraafikud – on väärtuslikud eelkõige kõrgemal tasemel tegutsevatele klubidele, jalgpalli-akadeemiatele ja suurklubidele, kellel on ressursid ja vajadus sellise analüüsi järele.

Praktilisel testimisel ilmnesid mitmed puudused, mis piiravad platvormi sobivust Eesti väiksemate klubide kontekstis. Esiteks on rollide eraldamine ebaselge – treeneril on sisuliselt administraatoriõigused, sealhulgas võimalus saata meilisõnumeid kõikidele klubi gruppidele, mitte ainult enda juhendatavale grupile. Sellise lahenduse puhul puudub selge rollipõhine piiritlemine, mis väiksemate klubide kontekstis võib viia tahtmatute andmete avalikustamiseni.

Teiseks on platvormi sõnumisaatmine piiratud: meilisõnumeid saab saata ainult lastele ja vanematele koos, kuid puudub võimalus saata personaalset infot eraldi ainult lastele või ainult vanematele. Näiteks olukorras, kus vanemad soovivad treeneriga arutada lapse arengut või käitumist treeningul, ei võimalda süsteem seda teha ilma, et laps ise seda sõnumit näeks. Selline piirang takistab konfidentsiaalset suhtlust, mis on noortejalgpalli kontekstis oluline.

Kolmandaks on mängijate ja laste andmete haldamine platvormi struktuuris segane – süsteem eraldab need kaheks kategooriaks, kuigi Eesti noortejalgpalli praktikas on tegemist ühede ja

samade isikutega. Lisaks ei võimalda süsteem mängu otse lisada, mis muudab treeningute ja mängude haldamise töövoos ebaefektiivseks.

[Joonis X: ProSoccerData mängu lisamise puudumine — lisa ekraanipilt]

Süsteemi funktsionaalsus hõlmab ulatuslikke scouting- ja meditsiinilisi andmevälju, mis on mõeldud suurte profiklubide ja akadeemiate vajadustele. Eesti väiksemate klubide kontekstis, kus tegutsevad vabatahtlikud treenerid ja klubide ressursid on piiratud, jääb selline funktsionaalsus kasutamata ning muudab süsteemi üledimensioneerituks. Eesti väiksemate klubide kontekstis võib ProSoccerData kasutamine olla piiratud selle spetsiifilise fookuse tõttu, kuna süsteem ei ole suunatud igapäevase haldus- ja planeerimistöö lihtsustamisele, vaid pigem sportliku soorituse analüüsile. Seetõttu jäävad tagaplaanile sellised praktilised vajadused nagu treeningväljakute kasutuse koordineerimine ja lihtne klubisisene infovahetus.

4.4 TeamSnap

TeamSnap on laialdaselt kasutatav meeskonnahalduse platvorm, mis on suunatud eelkõige harrastus- ja noortesportidele. Süsteem pakub vahendeid võistkondade haldamiseks, ajakavade koostamiseks ning liikmete ja lapsevanemate teavitamiseks [11].

TeamSnapi tugevuseks on selle kasutajasõbralikkus ja lihtne ligipääs põhifunktsionaalsusele. Platvorm toetab ajakavade jagamist, osalemiste kinnitamist ning sõnumite edastamist, mis muudab selle sobivaks suhteliselt lihtsate tööprotsessidega organisatsioonidele. Erinevalt Sportlyzerist ja ProSoccerDatast on rollide eraldamine TeamSnapis selgem ning süsteemi üldine keerukus madalam.

Samas on TeamSnapi piiranguks selle üldine ja universaalne lähenemine, mis ei arvesta spetsiifiliselt Eesti jalgpalliklubide töökorralduse ja ressursikasutuse eripärasid. Platvorm on suunatud eelkõige Põhja-Ameerika turule ning Eesti klubide seas ei ole see kasutust leidnud. Süsteem ei paku piisavalt paindlikke vahendeid mitme võistkonna ja treeningväljakute keskeks koordineerimiseks ning selle kohandatavus kohaliku konteksti jaoks on piiratud. Kasutajaliides ja tugimaterjal on ainult ingliskeelsed, mis võib takistada süsteemi kasutuselevõttu eestikeelsetes klubides, kus treenerid ja lapsevanemad ei pruugi inglise keelt piisaval tasemel vallata.

4.5 Analüüsi kokkuvõte ja järeldused

Peatükkides 4.2–4.4 kirjeldatud lahenduste analüüsi tulemuste süstematiseerimiseks koostati võrdlustabel (tabel 1), mis kajastab kaheteistkümnet funktsionaalset kriteeriumit. Kriteeriumid hõlmavad nii lahenduste üldist funktsionaalsust (maksete haldamine, osalemiste kinnitamine, mängijate statistika) kui ka Eesti väike- ja keskmise suurusega jalgpalliklubide spetsiifilisi vajadusi, mis tulenevad peatükis 2.1 kirjeldatud probleemist. Iga kriteeriumi puhul on märgitud, kas lahendus toetab vastavat funktsionaalsust (Jah) või mitte (Ei). Hinnangud põhinevad platvormi praktilisel testimisel.

Tabel 1. Olemasolevate lahenduste funktsionaalne võrdlus

Kriteerium	Sportlyzer	ProSoccerData	TeamSnap
Maksete haldamine	Jah	Ei	Jah
Osalemiste kinnitamine (RSVP)	Jah	Ei	Jah
Mängijate statistika ja analüüs	Jah	Jah	Ei
Eestikeelne kasutajaliides	Jah	Jah	Ei
Suunatud jalgpallile	Ei	Jah	Ei
Eestis kasutusel	Jah	Jah	Ei
Mängu otse kalendrisse lisamine	Jah	Ei	Jah
Treeningute nädalapõhine korduvus	Ei	Ei	Jah
Väljakute keskne koordineerimine	Ei	Ei	Ei
Grupipõhine sõnumisaatmine (ainult oma grupp)	Ei	Ei	Jah
Rollide selge eraldamine (treener ≠ admin)	Ei	Ei	Jah
Eraldi teavitus lastele ja vanematele	Ei	Ei	Ei
Täidetud kriteeriumid (12-st)	6/12	4/12	6/12

Võrdlusest ilmneb, et kõik analüüsitud platvormid pakuvad teatud funktsionaalsust, mis toetab spordiklubide tööd. Sportlyzer täidab kaheteistkümnest kriteeriumist viis – platvorm pakub eestikeelset liidest, mugavat maksete haldamist, osalemiste kinnitamist, mängijate statistikat ning mängude lisamist kalendrisse. TeamSnap täidab samuti viis kriteeriumit, pakkudes häid võimalusi treeningute korduvuseks, grupipõhiseks sõnumisaatmiseks, rollide eraldamiseks, osalemiste kinnitamiseks ning maksete haldamiseks. ProSoccerData täidab

neli kriteeriumit, eristudes eelkõige jalgpallispetsiifilise fookuse, mängijate analüütika ning eestikeelse liidese poolest.

Samas ilmneb tabelist selgelt, et just Eesti väiksemate jalgpalliklubide spetsiifilised vajadused jäävad kõigi lahenduste puhul katmata. Ükski platvorm ei toeta treeningväljakute keskset koordineerimist ega võimalda eraldi teavitust lastele ja vanematele – need on kaks kriteeriumit, mida ei täitnud ükski analüüsitud platvorm. Samuti on grupipõhise sõnumisaatmise ja rollide selge eraldamise toetus olemas ainult TeamSnapis, mis aga ei ole Eestis kasutusel ega paku eestikeelset kasutajaliidest.

Analüüsi põhjal võib tuvastatud puudused koondada kolme peamise probleemina. Esiteks ei paku ükski lahendus lihtsustatud ja Eesti kontekstile kohandatud liikmete ning rollide haldust, sest olemasolevad süsteemid on kas liiga keerukad või suunatud muudele eesmärkidele. Nagu testimisel ilmnas, on ProSoccerDatas rollide eraldamine ebaselge ning Sportlyzeris puudub grupipõhise infovahetuse konfidentsiaalsus. Teiseks puudub kõikidel analüüsitud lahendustel treeningväljakute ja ajakavade keskne koordineerimine klubitasandil, mis võimaldaks erinevate võistkondade ajakavasid hallata ühtses vaates. Kolmandaks on klubisisese infovahetuse toetamine ühtses süsteemis, sealhulgas osalemiste kinnitamine ja muudatustest teavitamine, olemasolevates lahendustes kas puudulik või seotud muu funktsionaalsusega, mitte eraldiseisev ja selge töövoog.

Lisaks mõjutavad lahenduste sobivust kohalikud eripärad, nagu klubide suurus, ressursid ja töökorraldus. Seetõttu võib järeldada, et olemasolevate lahenduste olemasolu ei välista vajadust kontekstipõhise ja sihipärase lahenduse järele. Tuvastatud puuduste põhjal sõnastatakse järgnevalt kavandatava süsteemi funktsionaalsed ja mittefunktsionaalsed nõuded.

4.6 Süsteemi funktsionaalsed nõuded

Infosüsteemide kavandamisel kirjeldavad funktsionaalsed nõuded, milliseid tegevusi ja töövooge süsteem peab toetama ning milline on selle käitumine kasutaja vaates [3]. Käesoleva süsteemi funktsionaalsed nõuded tulenevad Eesti jalgpalliklubide haldusprotsessidest ning eelmises alapeatükis tuvastatud puudustest olemasolevates

lahendustes. Nõuded on jaotatud kolme põhivaldkonna kaupa, mis vastavad süsteemi kesksele eesmärgile.

Liikmete ja võistkondade haldus:

- Süsteem võimaldab registreerida ja hallata klubi liikmeid, sealhulgas mängijaid, treenereid ja muid osapooli.
- Süsteem võimaldab luua ja hallata võistkondi ning määrata liikmetele kuuluvus konkreetsetes võistkondades.
- Süsteem võimaldab treeneril hallata oma võistkondade koosseise ning vaadata oma võistkonna liikmete andmeid.
- Süsteem võimaldab administraatoril hallata kogu klubi struktuuri, sealhulgas kasutajarolle ja võistkondade seoseid.
- Süsteem eraldab selgelt treenerite ja administraatorite rollid, treener näeb ja haldab ainult oma võistkonda, administraator haldab kogu klubi struktuuri.

Treeningute ja ressursside haldus:

- Süsteem võimaldab luua ja muuta treeningute ajakavasid ning siduda need konkreetsete võistkondade ja treeningväljakutega.
- Süsteem toetab treeningute nädalapõhist korduvust, treener saab määrata treeningu korduma igal nädalal samal päeval ja kellaajal.
- Süsteem võimaldab hallata treeningväljakute andmeid ja nende saadavust.
- Süsteem võimaldab klubitasandil näha treeningväljakute kasutust erinevate võistkondade lõikes, et vältida ajakavade kattuvust.
- Treeningu aja või koha muutmisel kajastub muudatus kõigile asjaosalistele.

Klubisisene teavitus ja osalemine:

- Süsteem võimaldab treeneril saata teavitusi oma võistkonna liikmetele.

- Süsteem võimaldab treeneril saata teavitusi eraldi lastele ja eraldi vanematele, tagades konfidentsiaalse suhtluse võimaluse.
- Sõnumisaatmine on grupipõhine, treener näeb ainult oma võistkonnaga seotud sõnumeid.
- Süsteem võimaldab liikmetel kinnitada osalemist treeningutel.
- Süsteem võimaldab treeneril näha osalemiste koondülevaadet enne treeningut.

4.7 Süsteemi mittefunktsionaalsed nõuded

4.7.1 Jõudlus ja skaleeritavus

Rakendus peab tagama süsteemi sujuva töö vähemalt 50 samaaegsele kasutajale ilma märgatava viivitusega. Lehekülgede laadimisaeg peab jääma alla 3 sekundi. Andmebaasipäringud optimeeritakse indeksite ja vahemälustamise abil, et vältida ülekoormust suurema kasutajahulga korral.

4.7.2 Kasutatavus ja responsiivsus

Kasutajaliides peab olema intuitiivne ning kasutatav ilma eraldi väljaõppeta. Rakendus peab olema täielikult responsiivne, toimides korrektselt nii lauaarvutis, tahvelarvutis kui nutitefonis. Liidese disain järgib Mobile First põhimõtet, kuna eeldatav peamine kasutajaseade on nutitefon — treenerid ja lapsevanemad kasutavad rakendust väljakul olles.

4.7.3 Autentimine ja autoriseerimine

Süsteem kasutab JWT-põhist (JSON Web Token) autentimist, mida haldab Supabase Auth teenus. Kasutajad logivad sisse e-posti ja parooliga. Paroolid salvestatakse räsituna bcrypt algoritmi abil, tagades, et isegi andmebaasi lekkimise korral ei ole paroolid loetavad.

Autoriseerimine on realiseeritud rollipõhiselt. Süsteemis on neli põhirolli: administraator, treener, mängija ja lapsevanem. Administraatoril on täielik ligipääs klubide, võistkondade, väljakute ja kasutajate haldamisele. Treener saab hallata oma võistkondi, planeerida treeninguid ja mängu ning jälgida kohalolekut. Mängija näeb oma treeninguid ja mängu ning saab kinnitada osalemist. Lapsevanem näeb oma lapse treeninguid, kinnitab osalemist ja saab teavitusi.

Lisaks rollipõhisele ligipääsule rakendatakse andmebaasi tasemel Row Level Security (RLS) poliitikat, mis tagab, et kasutaja näeb ja saab muuta ainult neid andmeid, millele tal on õigus. Näiteks näeb treener ainult oma võistkondade mängijaid ja lapsevanem ainult oma lapse andmeid.

4.7.4 Andmekaitse ja GDPR-i nõuded

Kuna rakendus töötleb isikuandmeid, sealhulgas alaealiste andmeid, peab süsteem vastama Euroopa Liidu isikuandmete kaitse üldmäärusele (GDPR, EL 2016/679). Jalgpalliklubi haldamise kontekstis on see eriti oluline, kuna süsteem töötleb mitmesuguseid isikuandmeid: identifitseerimisandmeid (nimi, e-posti aadress, telefoninumber), demograafilisi andmeid (sünniaeg, vanus), organisatsioonilisi andmeid (klubi, võistkond, roll süsteemis), tegevusandmeid (treeningul osalemine, mängude ajakava, kohaloleku ajalugu) ning perekondlikke seoseid (lapsevanema ja lapse vaheline seos).

GDPR-i artikkel 8 sätestab, et alla 16-aastaste laste isikuandmete töötlemiseks on vajalik vanema või eestkostja nõusolek. Kuna jalgpalliklubide noortevõistkonnad koosnevad valdavalt alaealistest, on see nõue süsteemi kontekstis keskse tähtsusega. Rakenduses on see lahendatud järgmiselt: alaealise konto on seotud lapsevanema kontoga ning lapsevanem annab registreerimisel nõusoleku lapse andmete töötlemiseks.

Süsteemi arendamisel järgitakse GDPR-i põhimõtteid. Andmete minimeerimise põhimõtte kohaselt kogutakse ainult klubi haldamiseks vajalikke andmeid — üleliigseid isikuandmeid, nagu aadress või isikukood, ei koguta. Eesmärgi piirangu põhimõtte kohaselt kasutatakse andmeid ainult klubi tegevuse korraldamiseks (treeningud, mängud, teavitused) ning neid ei edastata kolmandatele osapooltele. Turvalisuse põhimõte on tagatud JWT autentimise, bcrypt parooliräsamise, RLS-poliitikate ja HTTPS-krüpteeritud andmevahetuse kaudu. Läbipaistvuse põhimõtte kohaselt on kasutajal õigus teada, milliseid andmeid tema kohta kogutakse ja kuidas neid kasutatakse. Kasutajal on õigus oma profiiliandmeid muuta, konto kustutamise funktsionaalsus on planeeritud edasiarenduseks.

4.7.5 Turvameetmed

Lisaks autentimisele ja andmekaitsele rakendatakse süsteemis mitmeid tehnilisi turvameetmeid. Kogu suhtlus kliendi ja serveri vahel toimub HTTPS-protokolli kaudu, mis tagab andmete konfidentsiaalsuse ja tervikluse edastamisel.

SQL-süstimise rünnakute vältimiseks kasutatakse andmebaasipäringutes parameetrilisi päringuid Spring Data JPA ja Supabase klientteegi kaudu. XSS-rünnakute vastu kaitseb React raamistiku vaikimisi väljundi kodeerimine (output encoding), mis takistab pahatahtliku JavaScripti koodi süstimist lehekülgedesse. Tagarakenduses on seadistatud CORS-poliitika (Cross-Origin Resource Sharing), mis lubab päringuid ainult volitatud domeenidelt.

Kõik kasutaja sisendid valideeritakse nii eesrakenduse kui ka tagarakenduse poolel. Eesrakenduses toimub esmane valideerimine vormiväljadel, tagarakenduses kasutatakse Spring Boot valideerimismehhanisme (@Valid, @NotBlank jne). Kahepoolne valideerimine vähendab vigaste või pahatahtlike andmete jõudmist süsteemi.

Kokkuvõttes tagab kirjeldatud turvameetmete kombinatsioon, et rakendus kaitseb kasutajate isikuandmeid mitmel tasemel: autentimise, autoriseerimise, andmete krüpteerimise ja sisendvalideerimise kaudu. Alaealiste andmete kaitse on tagatud nii tehniliste meetmetega (RLS, rollipõhine ligipääs) kui ka organisatsiooniliste meetmetega (vanema nõusolek, andmete minimeerimine)..

5 Süsteemi analüüs ja arhitektuurilised valikud

Käesolevas peatükis kirjeldatakse arendatava süsteemi tehnilist ülesehitust ning põhjendatakse tehtud valikuid. Arhitektuurilised ja tehnoloogilised otsused tulenevad eelmises peatükis sõnastatud funktsionaalsetest ja mittefunktsionaalsetest nõuetest ning arvestavad prototüübi eesmärki ja ulatust.

5.1 Süsteemi arhitektuuri ülevaade

Käesoleva töö raames arendatav infosüsteem on kavandatud veebipõhise rakendusena, mille arhitektuur põhineb ees- ja tagarakenduse selgel eraldamisel. Selline arhitektuurne jaotus võimaldab käsitleda kasutajaliidest ja äriloogikat iseseisvate, kuid omavahel seotud komponentidena [5].

Süsteemi keskmes on tagarakendus, mis vastutab andmete halduse, ärireeglite ning rollipõhise ligipääsu eest. Rollipõhise ligipääsu vajadus tuleneb otseselt süsteemi mittefunktsionaalsest nõudest, mille kohaselt peavad administraatori ja treeneri õigused olema selgelt eristatud. Eesrakendus täidab kasutajaliidese rolli, pakkudes treeneritele ja klubi esindajatele visuaalset ja interaktiivset ligipääsu süsteemi funktsionaalsusele. Kasutajate ja süsteemi vaheline suhtlus toimub standardiseeritud rakendusliidese kaudu, mis võimaldab komponentide sõltumatut arendamist ja testimist.

Selline arhitektuur toetab prototüübi eesmärki, milleks on hinnata süsteemi sobivust Eesti jalgpalliklubide tööprotsesside toetamiseks, ilma et oleks vaja realiseerida täismahulist tootmissüsteemi.

[TODO] Lisada siia komponentdiagramm (Joonis X), mis näitab eesrakenduse, tagarakenduse, andmebaasi ja kasutaja vahelisi seoseid. draw.io

5.1.1 Süsteemi põhivood

Süsteemi arhitektuuriliste valikute põhjendamiseks on oluline kirjeldada, kuidas funktsionaalsetes nõuetes määratletud põhiprotsessid läbivad süsteemi erinevaid kihte. Tabelis 1 on esitatud neli keskset kasutusvoogu ning nende teekond läbi arhitektuuri komponentide.

Tabel 1. Süsteemi põhivood ja kaasatud komponendid

Põhivoog	Osalised rollid	Kaasatud komponendid	Tulemus
Treeningu loomine	Treener	Eesrakendus (vorm) → REST API → Tagarakendus (valideerimine, konflikti kontroll) → PostgreSQL	Treening salvestatud, seotud võistkonna ja väljakuga
Teavituse saatmine	Treener	Eesrakendus (teavituse koostamine) → REST API → Tagarakendus (rollipõhine filtreerimine) → PostgreSQL	Teavitus kättesaadav võistkonna liikmetele
Osalemise kinnitamine	Liige (tulevikus)	Eesrakendus (kinnituse nupp) → REST API → Tagarakendus (staatuse uuendamine) → PostgreSQL	Treener näeb osalemiste koondülevaadet
Väljakute koordineerimine	Administraator	Eesrakendus (kalendrivaade) → REST API → Tagarakendus (saadavuse kontroll) → PostgreSQL	Väljakute kasutus kuvatud ühtses vaates

Iga voo puhul algab kasutaja tegevus eesrakenduses, kust päring edastatakse REST API kaudu tagarakendusele. Tagarakendus kontrollib kasutaja rolli, valideerib sisendandmed ning teostab vajaliku äriloogika, sealhulgas ressursside konflikti kontrolli väljakute puhul. Tulemus salvestatakse PostgreSQL andmebaasi ning eesrakendus uuendab vaadet vastavalt tagastatud andmetele.

Näiteks treeningu loomise voo puhul sisestab treener eesrakenduses uue treeningu andmed (aeg, koht, võistkond). Tagarakendus kontrollib, kas valitud väljak on antud ajal saadaval, kas treeneril on õigus antud võistkonna treeninguid hallata, ning salvestab treeningu andmebaasi. Kui ilmneb ajakavade kattuvus, tagastab tagarakendus veateate ning eesrakendus kuvab selle treenerile.

[TODO] Lisa siia andmevoo diagramm (Joonis X), mis näitab treeningu loomise voogu läbi arhitektuuri kihtide: kasutaja → eesrakendus → REST API → tagarakendus (valideerimine, RBAC kontroll, konflikti kontroll) → andmebaas.

5.2 Eesarenduse ja tagarakenduse roll

Eesrakenduse peamine roll on kasutajaliidese realiseerimine ning kasutajate ja süsteemi vahelise suhtluse vahendamine. Käesoleva süsteemi puhul tähendab see treeningute ajakavade, võistkondade koosseisude ja teavituste kuvamist ning vastavate toimingute algamist kasutaja poolt. Need vaated tulenevad otseselt funktsionaalsetest nõuetest: liikmete ja võistkondade haldus eeldab koosseisude ning rollide kuvamist, treeningute haldus eeldab ajakavade ja väljakute kasutuse kuvamist ning teavituste töövoog eeldab osalemiste staatuste ja sõnumite esitamist. Eesrakendus ei sisalda äriloogikat, vaid edastab kasutaja tegevused struktureeritud päringutena tagarakendusele.

Tagarakendus vastutab süsteemi äriloogika, andmetöötluse ja turvalisuse eest. Sinna koondub reeglistik, mis määrab näiteks rollipõhised õigused, ressursikasutuse piirangud ning andmete tervikluse. Selline vastutuste jaotus vähendab süsteemi keerukust ning tagab, et kriitiline loogika ei sõltu kasutajaliidese teostusest [5].

Rollide selge eristamine on oluline mitme kasutajagrupiga süsteemide puhul [7]. Näiteks võib treeneril olla õigus luua ja muuta treeninguid ning saata teavitusi oma võistkonnale, samas kui administraatoril on õigus hallata kogu klubi struktuuri ning teha süsteemitasandi muudatusi. Need rollid vastavad funktsionaalsetes nõuetes kirjeldatud kasutajastenaariumidele.

5.3 Platvormi valik

Süsteemi realiseerimisel tuli valida, kas prototüüp luua klassikalise veebirakendusena või hübriidse mobiilirakendusena. Käesolevas töös käsitletakse veebirakendusena brauseris töötavat lahendust, mis on kasutatav erinevates seadmetes ja operatsioonisüsteemides ilma eraldi paigalduseta ning millele ligipääs toimub veebilehitseja kaudu [6]. Hübriidne mobiilirakendus tähendab seevastu mobiilirakendust, mida arendatakse ühe koodibaasina ning mis sihitakse mitmele operatsioonisüsteemile vastava raamistiku abil, kasutades enamasti WebView-põhist lähenemist [13].

Platvormi valikul lähtuti eelkõige prototüübi eesmärgist ja mittefunktsionaalsest nõudest, mille kohaselt peab süsteem olema kasutatav veebilehitsejas ilma eraldi tarkvara paigaldamiseta. Kuigi mobiilirakendus sobitub lõppkasutajate igapäevase

kasutuskeskkonnaga, on käesoleva töö fookus suunatud eeskätt treenerite ja klubi esindajate haldus- ja planeerimisülesannetele.

5.3.1 Veebirakenduse sobivus prototüübiks

Veebirakenduse kasutamine on jalgpalliklubi haldussüsteemi kontekstis põhjendatud eelkõige selle laia kättesaadavuse ning sobivuse haldus- ja planeerimisülesannete täitmiseks. Süsteemi peamisteks kasutajateks on treenerid ja klubi administratiivtöötajad, kelle töö hõlmab treeningute planeerimist, võistkondade koosseisude haldamist ning treeningväljakute ja aegade koordineerimist. Need tegevused eeldavad paralleelselt mitme andmeüksuse käsitlemist, mis on praktikas mugavam suuremal ekraanil [14].

Lisaks on veebirakenduse eeliseks lihtne kasutuselevõtt: rakenduse kasutamiseks puudub vajadus eraldi paigalduseks ning ligipääs toimub veebiaadressi kaudu. Prototüübi kontekstis on oluline ka tsentraalne uuendamine, kus parandused ja uued funktsionaalsused rakenduvad serveripoolselt ning jõustuvad koheselt kõigile kasutajatele.

5.3.2 Hübriidmobiilirakenduse piirangud

Hübriidne mobiilirakendus võib olla kasutajasõbralik eelkõige lõppkasutajatele, kes tarbivad infot ja täidavad lihtsamaid toiminguid, nagu osalemise kinnitamine või teavituste lugemine [13]. Käesoleva prototüübi puhul ei ole mängijapoolne kasutusmugavus siiski määrav platvormivaliku kriteerium, kuna eesmärk on eeskätt haldus- ja planeerimisvoogude katsetamine. WebView-põhise arhitektuuri tõttu võib kasutajaliidese sujuvus olla nõrgem ning pluginate kasutamine suurendab hoolduse keerukust [13]. Mobiilirakendus võiks edasises arenduses pakkuda lisaväärtust eelkõige reaajas push-teavituste ja kiire osalemise kinnitamise kaudu.

5.3.3 Valiku tulemus

Läbiviidud analüüsi tulemusel osutus prototüübi jaoks sobivaimaks lahenduseks veebirakendus. Veebipõhine lahendus tagab lihtsa ja platvormiülese ligipääsu, välistab paigaldusvajaduse ning võimaldab süsteemi kiiret testimist ja kasutuselevõttu [6], [13]. Tabelis 2 on esitatud platvormi võrdlus.

Tabel 2. Platvormi võrdlus

Võrdlusalus	Veebirakendus	Hübriid mobiilirakendus
Ligipääsetavus	Kasutatav erinevates seadmetes veebilehitseja kaudu	Vajab rakenduse paigaldamist rakendustepoe kaudu
Kasutuselevõtu keerukus	Madal, ligipääs veebiaadressi kaudu	Kõrgem, paigaldus ja platvormipõhine haldus
Sobivus haldusülesanneteks	Väga hea, toetab paralleelset andmekäsitlust suuremal ekraanil	Piiratud, väiksem ekraan raskendab keerukaid voogusid
Kasutajagrupi sobivus	Sobib treeneritele ja klubi administratiivtöötajatele	Sobib eelkõige lõppkasutajatele (mängijad, lapsevanemad)
Arenduse kiirus	Kõrge, muudatused rakenduvad kohe	Piiratum, sõltub rakenduse levitusest
Sobivus prototüübi eesmärgiga	Väga sobiv	Vähem sobiv prototüübi faasis

5.4 Tagarakenduse tehnoloogia valik

Tagarakenduse raamistiku valimiseks kasutati Trade-Off Analysis (TOA) meetodit, mis võimaldab struktureeritult võrrelda alternatiive mitmete kaalutud kriteeriumide alusel [26]. Meetod koosneb seitsmest sammust: eesmärgi määratlemine, alternatiivide tuvastamine, valikukriteeriumide sõnastamine, kriteeriumide kaalumine, alternatiivide hindamine, kaalutud skoori arvutamine ning tulemuste analüüs.

Analüüsi eesmärk on valida tagarakenduse raamistik, mis katab peatükis 4 sõnastatud funktsionaalsed ja mittefunktsionaalsed nõuded ning sobib prototüübi realiseerimiseks. Kaaluti kolme raamistikku: Java Spring Boot, C# ASP.NET Core ja Node.js Express.js.

5.4.1 Valikukriteeriumid ja kaalud

Valikukriteeriumid tulenevad süsteemi nõuetest ning prototüübi praktilistest piirangutest. Iga kriteeriumi kaal (1 kuni 5) väljendab selle olulisust käesoleva töö kontekstis.

Rollipõhise ligipääsu tugi (kaal 5). Süsteemi mittefunktsionaalne nõue eeldab administraatori ja treeneri õiguste selget eristamist. See on süsteemi üks keskseid nõudeid, mistõttu on kriteeriumi kaal maksimaalne.

Ökosüsteemi terviklikkus (kaal 5). Prototüübi kontekstis on oluline, et põhifunktsionaalsus (autoriseerimine, andmepöördus, REST API) oleks teostatav ühe raamistiku piires ilma suure hulga kolmandate osapoolte teکیدeta. See vähendab sõltuvuste arvu ja hoolduskeerukust.

Arhitektuurne kihistus (kaal 4). Selge vastutuste jaotus (kontroller, teenus, andmepöördus) tagab koodi hooldatavuse ja laiendatavuse. Kaal on kõrge, kuna prototüüp peab olema aluseks edasisele arendusele.

Kogukonna suurus ja dokumentatsioon (kaal 3). Suurem kogukond tähendab rohkem näiteid, ja õppematerjaale, mis kiirendab probleemide lahendamist. Spring Bootil on GitHubis ligikaudu 80 000 tähti [28], Express.js-il 66 000 [30] ja ASP.NET Core'il 37 700 [29].

Õppimiskõver ja arenduskiirus (kaal 3). Prototüübi arendamiseks on piiratud aeg, mistõttu on oluline, et autoril oleks võimalik raamistikuga tõhusalt töötada. Varasem kogemus vähendab õppimisaega.

Jõudlus (kaal 2). TechEmpower Framework Benchmarks Round 23 tulemused näitavad raamistike vahelisi jõudluserinevusi [27]. Kaal on madal, kuna süsteemi sihtgrupp on väikesed ja keskmise suurusega Eesti jalgpalliklubid, kus samaaegsete kasutajate arv jääb eeldatavalt alla 500. Sellise koormuse puhul suudavad kõik kolm raamistikku päringuid teenindada ilma märgatavate erinevusteta.

5.4.2 Alternatiivide hindamine

Spring Boot (Java). Spring Security pakub sisseehitatud rollipõhist autoriseerimist annotatsioonide `@PreAuthorize` ja `@Secured` kaudu [16]. Ökosüsteem katab kõik põhivajadused: Spring Security autoriseerimiseks, Spring Data JPA andmepöörduseks, Spring Boot Starter Web REST API realiseerimiseks [15]. Raamistik kehtestab selge MVC kihtstruktuuri [5]. Autoril on selle raamistikuga varasem kogemus. Jõudlus TechEmpower testides keskmine [27].

ASP.NET Core (C#). ASP.NET Identity toetab rollipõhist autoriseerimist [21]. Raamistik pakub Entity Framework Core andmepöörduseks ja sisseehitatud turvalahendusi. Arhitektuur on selgelt kihistatud sarnaselt Spring Bootile. Autoril on piiratud varasem kogemus C# keelega, kuid .NET ökosüsteemi tundmine on vähene. Jõudlus TechEmpower testides kolmest kõrgeim [27].

Express.js (Node.js). Rollipõhise ligipääsu teostamiseks tuleb kasutada kolmandate osapoolte teeke (Passport.js). Samaväärseks funktsionaalsuseks on vajalik mitmete eraldi teekide integreerimine (Passport.js, Sequelize/TypeORM, Helmet.js, cors) [24]. Raamistik ei kehtesta kindlat arhitektuurset mustrit. JavaScript on autorile tuttav, kuid RBAC teostamine keerukam. Jõudlus TechEmpower testides keskmine kuni kõrge [27].

5.4.3 Trade-off analüüsi tulemus

Tabelis 3 on esitatud kaalutud hindamise koondtulemus. Iga alternatiivi hinnati skaalal 1 (väga nõrk) kuni 5 (väga tugev) iga kriteeriumi lõikes. Kaalutud skoor on kriteeriumi kaalu ja hinde korrutis.

Todo: lisada autori kogemus

Tabel 3. Tagarakenduse raamistike Trade-Off Analysis

Kriteerium	Kaal	Spring Boot		ASP.NET Core		Express.js	
		Hinne	Skoor	Hinne	Skoor	Hinne	Skoor
Rollipõhise ligipääsu tugi	5	5	25	4	20	2	10
Ökosüsteemi terviklikkus	5	5	25	4	20	2	10
Arhitektuurne kihistus	4	5	20	5	20	2	8
Kogukonna suurus ja dokumentatsioon	3	4	12	3	9	5	15
Õppimiskover ja arenduskiirus	3	4	12	2	6	4	12
Jõudlus	2	3	6	5	10	4	8
Kokku	22		100		85		63

Analüüsi tulemusel saavutas Spring Boot kõrgeima kaalutud skoori (100 punkti), edestades ASP.NET Core'i (85) ja Express.js-i (63). Spring Booti eelis tuleneb eeskätt kahest kõrgeima kaaluga kriteeriumist: rollipõhise ligipääsu sisseehitatud tugi ning ökosüsteemi terviklikkus, kus kõik vajalikud komponendid on saadaval ühe raamistiku piires.

ASP.NET Core on tehniliselt võrdväärne alternatiiv, mis edestab Spring Booti jõudluse kriteeriumis. Kui jõudluse kaal oleks kõrgem, näiteks suuremahulise tootmissüsteemi puhul, muutuks vahe Spring Booti ja ASP.NET Core'i vahel väiksemaks. Käesoleva prototüübi

kontekstis on jõudluse kaal siiski madal ning ASP.NET Core'i peamine piirang on autori vajadus uut ökosüsteemi tundma õppida, mis pikendaks arendusaega.

Express.js jääb maha eelkõige sisseehitatud funktsionaalsuse puudumise tõttu. Kuigi raamistik sobib hästi kiireks prototüüpimiseks, nõuab rollipõhise ligipääsu, struktureeritud arhitektuuri ja turvalahenduste teostamine märkimisväärset lisatööd kolmandate osapoolte teekide integreerimisel.

5.5 Eesrakenduse tehnoloogia valik

Eesrakenduse raamistiku valimiseks kasutati sama TOA meetodit [26]. Kaaluti kolme levinumat JavaScripti raamistikku: React, Angular ja Vue.js.

5.5.1 Valikukriteeriumid ja kaalud

Komponendimudeli paindlikkus (kaal 5). Süsteemi funktsionaalsed nõuded eeldavad mitmekesiseid vaateid: liikmete nimekirjad, treeningute kalendrivaated, teavituste töövood. Raamistik peab võimaldama neid vaateid realiseerida paindlikult ja korduvkasutatavalt. Kaal on maksimaalne, kuna kasutajaliidese realiseerimine on eesrakenduse keskne ülesanne.

Ökosüsteem ja valmiskomponendid (kaal 4). Kolmandate osapoolte komponentide (kalendrivaated, tabelid, vormid) kättesaadavus kiirendab prototüübi arendust. Reactil on npm-is üle 2,5 miljoni seotud paketi, Angularil ligikaudu 350 000 ja Vue.js-il ligikaudu 500 000 [31].

Õppimiskõver ja arenduskiirus (kaal 4). Prototüübi piiratud ajaraam eeldab, et raamistikuga saab kiiresti tööle hakata. Vue.js on kolmest madalaima õppimiskõverega, React keskmine ja Angular kõrgeima [22], [23].

Kogukonna suurus (kaal 3). GitHubi tärnide põhjal on React ligikaudu 234 000 ja Vue.js ligikaudu 208 000 suurima kogukonnaga, Angular ligikaudu 97 000 jääb neist märgatavalt maha [31]. Suurem kogukond tähendab rohkem õppematerjale ja kiiremat probleemide lahendust.

Rakenduse paketi suurus (kaal 2). Väiksem paketi suurus kiirendab rakenduse laadimist kasutaja seadmes. Vue.js paketi suurus on ligikaudu 33 KB, Reactil 42 KB ja Angularil 143 KB (gzip) [31]. Kaal on madal, kuna prototüübi kontekstis ei ole laadimiskiirus kriitiline.

5.5.2 Alternatiivide hindamine

React. Meta Platforms arendatud JavaScripti teek, mis toetab komponendipõhist lähenemist [17]. React ei kehtesta ranget rakenduse struktuuri, vaid jätab arendajale vabaduse valida sobiv arhitektuur. Reacti kaasaegne arendusmudel hoiab koodi kompaktsena ja olekuhalduse läbipaistvana. Prototüübi vaates on Reacti paindlikkus eelis, sest see võimaldab kiiret iteratsiooni ja vaadete lisamist. Turvalisuse vaates aitab korrektne sisendi käsitlemine vähendada tüüpilisi kasutajaliidese riske [18]. Autoril on varasem kogemus Reactiga.

Angular. Google'i arendatud täisraamistik, mis kehtestab rakendusele kindla struktuuri ning pakub sisseehitatud lahendusi marsruutimiseks, olekuhalduseks ja sõltuvussüstimiseks [22]. Angular kasutab TypeScripti ning klassipõhist komponentmudelit. Prototüübi kontekstis on Angulari piiranguks selle kõrge õppimiskõver ning struktureeritus, mis lisab arendusele keerukust. Dekoraatorid, moodulid ja sõltuvussüstimine lisavad koodi mahtu ka lihtsate vaadete puhul. Arendajal puudub varasem kogemus Angulariga.

Vue.js. Progressiivne JavaScripti raamistik, mis pakub intuitiivset template-süntaksit ja Single File Components mudelit [23]. Vue.js õppimiskõver on kolmest valikust madalaim ning see sobib hästi väiksemate projektide kiireks prototüüpimiseks. Samas on selle ökosüsteem väiksem kui Reactil, mis võib piirata kolmandate osapoolte komponentide kättesaadavust. Arendajal puudub varasem kogemus Vue.js-iga.

5.5.3 Trade-off analüüsi tulemus

Tabelis 4 on esitatud eesrakenduse raamistike kaalutud hindamise koondtulemus.

Tabel 4. Eesrakenduse raamistike Trade-Off Analysis

Kriteerium	Kaal	React		Angular		Vue.js	
		Hinne	Skoor	Hinne	Skoor	Hinne	Skoor
Komponendimudeli paindlikkus	5	5	25	3	15	4	20
Ökosüsteem ja valmiskomponendid	4	5	20	4	16	3	12
Õppimiskover ja arenduskiirus	4	4	16	2	8	5	20

Kogukonna suurus	3	5	15	4	12	4	12
Rakenduse paketi suurus (bundle)	2	3	6	2	4	5	10
Kokku	18		82		55		74

Analüüsi tulemusel saavutas React kõrgeima kaalutud skoori (82 punkti), edestades Vue.js-i (74 punkti) ja Angulari (55 punkti). Reacti eelis tuleneb eelkõige komponendimudeli paindlikkusest ning ökosüsteemi laiuusest, mis võimaldab kasutada valmiskomponente kalendrivaadete, tabelite ja vormide realiseerimiseks.

Vue.js on lähim alternatiiv, mis saavutab Reactist kõrgema skoori õppimiskõvere ja paketi suuruse kriteeriumides. Autoril puudub siiski varasem kogemus Vue.js-iga ning selle ökosüsteem on Reacti omast väiksem, mistõttu jääb Vue.js käesoleva prototüübi kontekstis teisele kohale.

Angular jääb maha eelkõige kõrge õppimiskõvere tõttu. Täisraamistiku struktureeritus, mis on eelis suuremahuliste ettevõtte rakenduste puhul, muutub prototüübi kontekstis piiranguks, kuna lisab arendusele keerukust, mis ei ole prototüübi mahus põhjendatud.

5.6 Andmebaas ja konteineripõhine keskkond

5.6.1 Andmebaasi valik

Andmebaasi valimiseks rakendati eelnevate komponentidega sama TOA meetodit [26]. Võrdlusesse kaasati kolm levinud andmebaasisüsteemi: PostgreSQL, MySQL ja MongoDB.

Relatsiooniliste seoste tugi (kaal 5). Süsteemi funktsionaalsed nõuded eeldavad omavahel seostatud andmeüksuste haldamist: klubid, võistkonnad, liikmed, rollid, treeningud, väljakud ja osalemisstaatused. Nende puhul on oluline tagada viidete terviklus ja piirangutel põhinevad reeglid. Kaal on maksimaalne, kuna andmete relatsioonilisus on süsteemi arhitektuuri alus.

Andmetüüpide paindlikkus (kaal 3). PostgreSQL toetab lisaks standardsetele SQL tüüpidele ka JSON, massiive ja kohandatud tüüpe, mis võib osutuda kasulikuks edasisel arendamisel

[19]. MySQL toetab põhilisi SQL tüüpe. MongoDB kasutab dokumendipõhist mudelit, mis on paindlik, kuid ei sobi hästi relatsiooniliste seoste jaoks.

Spring Boot integratsioon (kaal 4). Kuna tagarakendus realiseeritakse Spring Bootiga, on oluline, et andmebaas integreerub Spring Data JPA kaudu süjuvalt. PostgreSQL ja MySQL on mõlemad Spring Data JPA poolt hästi toetatud. MongoDB eeldab eraldi Spring Data MongoDB mõõdulit, mis ei toeta JPA standardset repositooriumi mustrit.

Kogukond ja dokumentatsioon (kaal 3). Kõik kolm andmebaasi on laialt kasutatavad ja hästi dokumenteeritud. DB-Engines reitingu järgi on PostgreSQL ja MySQL maailma populaarseimate andmebaaside seas esiviisikus [32].

Avatud lähtekood ja tasuta kasutus (kaal 3). Kõik kolm on avatud lähtekoodiga ja tasuta kasutatavad, mistõttu see kriteerium alternatiive ei erista.

Tabelis 5 on esitatud andmebaasi valiku Trade-Off Analysis tulemus.

Tabel 5. Andmebaasisüsteemide Trade-Off Analysis

Kriteerium	Kaal	PostgreSQL		MySQL		MongoDB	
		Hinne	Skoor	Hinne	Skoor	Hinne	Skoor
Relatsiooniliste seoste tugi	5	5	25	5	25	1	5
Andmetüüpide paindlikkus	3	5	15	3	9	4	12
Spring Boot integratsioon	4	5	20	5	20	3	12
Kogukond ja dokumentatsioon	3	5	15	5	15	4	12
Avatud lahtekood ja tasuta kasutus	3	5	15	5	15	5	15
Kokku	18		90		84		56

Analüüsi tulemusel saavutas PostgreSQL kõrgeima kaalutud skoori (90 punkti), edestades MySQLi (84) ja MongoDB-d (56). PostgreSQL ja MySQL on relatsiooniliste seoste ja Spring Boot integratsiooni poolest võrdväärsed. PostgreSQLi eelis tuleneb andmetüüpide

suuremast paindlikkusest (JSON tugi, massiivid, kohandatud tüübid), mis loob parema aluse edasiseks arenduseks [19].

MongoDB jääb maha eelkõige relatsiooniliste seoste puuduliku toe tõttu. Dokumendipõhine mudel ei sobi süsteemi konteksti, kus andmete vahel on selged seosed (näiteks võistkond kuulub klubisse, treening on seotud väljakuga) ning viidete terviklus on oluline.

5.6.2 Docker

Arendus- ja testkeskkonna ühtlustamiseks valiti Dockeril põhinev konteineripõhine lähenemine. Docker võimaldab pakendada rakenduse sõltuvused ja teenused standardiseeritud keskkonnaks, mis vähendab erinevusi arendaja arvutite konfiguratsioonide vahel ning muudab prototüübi ülesseadmise korduvaks ja kontrollitavaks [20]. Docker Compose võimaldab defineerida kogu süsteemi (tagarakendus, eesrakendus, andmebaas) ühe konfiguratsioonifailina ning käivitada kõik teenused ühe käsuga.

Konteinerite kasutamine toetab ka iteratiivset arendust ja testimist. Samas käsitletakse Dockeri kasutamist käesolevas töös eeskätt arendus- ja prototüübi demonstreerimise kontekstis; tootmiskeskonna turvaseadistused, monitooring ja kõrge kättesaadavuse lahendused jäävad käesoleva töö käsitlusalast välja.

5.7 Tehnoloogiate ja arhitektuuri kokkuvõte

Käesolevas peatükis tehtud arhitektuurilised ja tehnoloogilised valikud tulenevad peatükis 4 sõnastatud nõuetest ning toetavad töö põhieesmärki: luua ja hinnata veebipõhist prototüüpi, mis sobitub Eesti jalgpalliklubide tüüpiliste haldus- ja planeerimisprotsessidega. Kõik tehnoloogiavalikud on tehtud Trade-Off Analysis meetodil [26], kus kriteeriumide kaalud tulenevad süsteemi funktsionaalsetest ja mittefunktsionaalsetest nõuetest. Tabelis 6 on esitatud valitud tehnoloogiate koondülevaade.

Tabel 6. Valitud tehnoloogiate koondülevaade

Komponent	Valitud tehnoloogia	Peamine alternatiiv
Tagarakendus	Spring Boot (Java)	ASP.NET Core (85), Express.js (63)
Eesrakendus	React	Vue.js (74), Angular (55)
Andmebaas	PostgreSQL	MySQL (84), MongoDB (56)
Konteinerikeskkond	Docker	Vagrant, käsitsi seadistamine
Suhtlusprotokoll	REST API (JSON)	GraphQL, gRPC

Eesrakenduse ja tagarakenduse eraldamine loob selge vastutuse jaotuse ning võimaldab kasutajaliidest ja äriloogikat arendada ning testida sõltumatult [5]. Veebipõhine platvormivalik vähendab kasutuselevõtu barjääri ning sobib prototüübi puhul eriti hästi treenerite ja klubi esindajate töövoogude katsetamiseks [6], [13], [14].

Iga tehnoloogia valik on põhjendatud konkreetsete alternatiivide kaalutud võrdluse kaudu ning arvestab nii funktsionaalseid nõudeid kui ka prototüübi praktilist teostamise konteksti. Valitud arhitektuur ei taotle maksimaalset tehnilist keerukust ega tootmiskeskonna täielikku käsitlust, vaid loob tasakaalu arenduse mahu ja süsteemi valideerimiseks vajaliku funktsionaalsuse vahel, mis on bakalaureusetöö raames põhjendatud.

6 Arendustöö

[TODO] See peatükk kirjeldab prototüübi praktilist realiseerimist. Alljärgnevalt on toodud soovituslik struktuur ja juhised iga alapeatüki kirjutamiseks.

6.1 Arendusprotsessi korraldus

[TODO] Kirjelda, kuidas arendustöö oli organiseeritud:

[TODO] • Millist töövoogu kasutasid (nt iteratiivne arendus, Scrum-põhine sprint)?

[TODO] • Versioonihaldus: Git kasutamine, haru strateegia (feature branches?)

[TODO] • Projekti haldusvahendid: Jira, Trello vms kasutamine ülesannete jälgimiseks

[TODO] • Arenduskeskkond: IntelliJ IDEA (Spring Boot), VS Code (React), Docker Compose konfiguratsioon

[TODO] • Mis järjekorras arendus toimus: andmebaas → tagarakendus → eesrakendus?

6.2 Andmebaasi mudel

[TODO] Kirjelda andmebaasi ülesehitust:

[TODO] • Lisa ER-diagramm (andmebaasi olemite suhtediagramm)

[TODO] • Kirjelda kõik põhiolemid: Club, Team, Member, User, TrainingSession, Field, Attendance, Notification

[TODO] • Selgita olemite vahelisi seoseid: Club→Team (1:n), Team→Member (n:m), TrainingSession→Field (n:1), jne

[TODO] • Põhjenda tabelite disaini otsuseid, miks just selline struktuur? Kuidas see toetab funktsionaalseid nõudeid?

[TODO] • Näita konkreetseid näiteid (nt SQL tabelidefinitioone või JPA olemeid)

6.3 Tagarakenduse realiseerimine

[TODO] Kirjelda Spring Boot tagarakenduse ülesehitust:

- [TODO] • *Projekti struktuur: controller, service, repository, model, config kihid*
- [TODO] • *REST API disain: milliseid endpoint'e realiseeriti ja millistele nõuetele need vastavad*
- [TODO] • *Rollipõhise ligipääsu realiseerimine Spring Security abil: kuidas kontrollitakse administraatori ja treeneri õigusi*
- [TODO] • *Autentimine: JWT token'id, sessioonihaldus*
- [TODO] • *Ärireeglite rakendamine: nt väljakute konflikti kontroll, osalemise kinnitamise loogika*
- [TODO] • *Lisa koodinäiteid (nt controller klass, security config)*

6.4 Eesrakenduse realiseerimine

- [TODO] *Kirjelda React eesrakenduse ülesehitust:*
- [TODO] • *Komponendipuu struktuur: pages, components, services, context*
- [TODO] • *Põhivaated: sisselogimine, armatuurlaud, võistkondade haldus, treeningute kalender, teavitused*
- [TODO] • *Marsruutimine ja rollipõhine liidese kohandamine: mida näeb treener vs administraator*
- [TODO] • *API suhtlus: kuidas eesrakendus pärib andmeid tagarakendusest (fetch/axios)*
- [TODO] • *Lisa ekraanipildid või joonised peamistest vaadetest*

6.5 Konteineripõhine keskkond

- [TODO] *Kirjelda Docker konfiguratsiooni:*
- [TODO] • *docker-compose.yml ülesehitus: milliseid teenuseid jooksutatakse*
- [TODO] • *Kuidas käivitada kogu süsteem ühe käsuga*
- [TODO] • *Keskkonnamuutujate haldus*

7 Valminud rakendus ja hindamine

[TODO] See peatükk esitleb valminud prototüüpi ja hindab selle sobivust.

7.1 Prototüübi ülevaade

[TODO] Esita valminud süsteemi ülevaade:

[TODO] • Lisada ülevaatlik süsteemipilt (deployment diagram või high-level architecture joonis)

[TODO] • Kirjeldada, milline on süsteemi lõplik funktsionaalsus

[TODO] • Lisada ekraanipildid põhivoogudest: sisselogimine → armatuurlaud → treeningu loomine → teavitus → osalemine

7.2 Funktsionaalse sobivuse hindamine

[TODO] Hinnata, kas prototüüp täidab funktsionaalsed nõuded:

[TODO] • Koostada tabel: iga funktsionaalne nõue (ptk 4.6) vs teostatud/osaliselt/ei

[TODO] • Kirjeldada, millised nõuded on täidetud ja millised mitte ning miks

[TODO] • Vastata uurimisülesandele: mil määral koondab prototüüp kolm haldusprotsessi ühte keskkonda?

7.3 Piirangud ja edasiarendusvõimalused

[TODO] Kirjelda ausalt prototüübi piirangud:

[TODO] • Mis jäi realiseerimata ja miks?

[TODO] • Tootmiskeskkonna nõuded, mis ei olnud fookuses (HA, monitooring, koormustestid)

[TODO] • Mobiilirakenduse lisamine tulevikus (push-teavitused, kiire osalemine)

[TODO] • Mängija/lapsevanema rolli lisamine

[TODO] • Mitmekeelsus (eesti, inglise, vene keel)

Kokkuvõte

[TODO] Kirjuta kokkuvõte (1 kuni 1,5 lehekülge):

[TODO] • *Esimene lõik: mis oli probleem ja töö eesmärk*

[TODO] • *Teine lõik: mida tehti, analüüsimise lahendusi, kavandati ja realiseeriti prototüüp*

[TODO] • *Kolmas lõik: peamised tulemused, mida prototüüp näitas, kas eesmärk saavutati*

[TODO] • *Neljas lõik: piirangud ja edasiarendus*

[TODO] • *NB! Kokkuvõttes ei ole uut infot, viiteid ega sissejuhatust. Ainult järeldused.*

Kasutatud kirjandus

- [1] Eesti Spordiregister. Jalgpall – harrastajate ja klubide statistika. [15.01.2026]. url: <https://www.spordiregister.ee>
- [2] A. R. Hevner, S. T. March, J. Park, S. Ram. Design Science in Information Systems Research. MIS Quarterly, 28(1), 75–105, 2004.
- [3] I. Sommerville. Software Engineering, 10th Global Edition. Pearson, 2016.
- [4] K. C. Laudon, J. P. Laudon. Management Information Systems: Managing the Digital Firm, 16th Edition. Pearson, 2020.
- [5] M. Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.
- [6] Mozilla Developer Network. Progressive web apps. [15.01.2026]. url: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- [7] R. Sandhu, E. Coyne, H. Feinstein, C. Youman. Role-Based Access Control Models. IEEE Computer, 29(2), 38–47, 1996.
- [8] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn. Proposed NIST Standard for Role-Based Access Control. ACM Transactions on Information and System Security, 4(3), 224–274, 2001.
- [9] Sportlyzer. Club Management Software. [15.01.2026]. url: <https://sportlyzer.com>
- [10] ProSoccerData. Football Club Management Platform. [15.01.2026]. url: <https://prosoccerdata.com>
- [11] TeamSnap. Sports Team Management Software. [15.01.2026]. url: <https://www.teamsnap.com>
- [12] Startupbeat. Sportlyzer startup profile. [15.01.2026]. url: <https://startupbeat.com>
- [13] Interaction Design Foundation. Web Apps vs Native Apps vs Hybrid Apps. [15.01.2026]. url: <https://www.interaction-design.org/literature/article/web-apps-vs-native-apps-vs-hybrid-apps>
- [14] Hostinger. What Is a Web Application? [15.01.2026]. url: <https://www.hostinger.com/tutorials/what-is-web-application>

- [15] Spring. Spring Boot Reference Documentation. [15.01.2026]. url: <https://spring.io/projects/spring-boot>
- [16] Spring. Spring Security Architecture. [15.01.2026]. url: <https://spring.io/projects/spring-security>
- [17] Meta Platforms. React – Main Concepts. [15.01.2026]. url: <https://react.dev>
- [18] OWASP Foundation. Cross Site Scripting Prevention Cheat Sheet. [15.01.2026]. url: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Scripting_Prevention_Cheat_Sheet.html
- [19] PostgreSQL Global Development Group. PostgreSQL Documentation. [15.01.2026]. url: <https://www.postgresql.org/docs/>
- [20] Docker, Inc. Docker Documentation. [15.01.2026]. url: <https://docs.docker.com/>
- [21] Microsoft. ASP.NET Core Identity. [15.01.2026]. url: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>
- [22] Google. Angular Documentation. [15.01.2026]. url: <https://angular.dev>
- [23] Vue.js. Vue.js Documentation. [15.01.2026]. url: <https://vuejs.org>
- [24] OpenJS Foundation. Express.js Documentation. [15.01.2026]. url: <https://expressjs.com>
- [25] Eesti Spordiregister. Harrastajate statistika. [15.01.2026]. url: <https://www.spordiregister.ee/et/statistika?module=har>
- [26] T. Robal. Trade-Off Analysis. IAS0320 Computer Systems Engineering, Lecture 6. Tallinn University of Technology, 2023.
- [27] TechEmpower. Framework Benchmarks Round 23. [15.02.2025]. url: <https://www.techempower.com/benchmarks/>
- [28] Spring Projects. Spring Boot GitHub Repository. [15.01.2026]. url: <https://github.com/spring-projects/spring-boot>
- [29] Microsoft. ASP.NET Core GitHub Repository. [15.01.2026]. url: <https://github.com/dotnet/aspnetcore>

- [30] OpenJS Foundation. Express.js GitHub Repository. [15.01.2026]. url: <https://github.com/expressjs/express>
- [31] npm, Inc. npm trends: React vs Angular vs Vue. [15.01.2026]. url: <https://npmtrends.com/angular-vs-react-vs-vue>
- [32] DB-Engines. DB-Engines Ranking. [15.01.2026]. url: <https://db-engines.com/en/ranking>